

Contents

1	Volume ParaView/VTK Output Path — Exploration (free-surface slice prep)	1
1.1	Overview	1
1.2	Architecture / data flow	1
1.3	Free surface identity	2
1.4	SubMesh transfer feasibility (verified against MFEM source)	2
1.5	Restart	3
1.6	Gotchas	3
1.7	Open questions (for the plan / user)	3

1 Volume ParaView/VTK Output Path – Exploration (free-surface slice prep)

Scope: how `spatial_dyn_driver.cpp` writes volume VTK/PVD/VTKHDF output, so we can add a **free-surface** ($z = z_{\text{max top}}$) slice extracted from those same volume fields. Default for the new slice output: **ON**.

1.1 Overview

The driver owns up to two volume `seas::ParaViewOutput<ParMesh>` collections plus a separate fault-surface artefact:

Collection	Object	Mesh	Fields	Default
volume	pv_out	full pmesh	velocity (L2-p, 3-comp), mpi_rank (L2-p0), 8 SAFS static fault params, + 8 fault projections	OFF (paraview_vol_ume="off")
ParaView_bulk/stress	pv_bulk_out	full pmesh	sigma_xx..sigma_xz ($6 \times$ L2-p scalar)	OFF (paraview_bulk="off")
fault surface	(inside pv_out)	gather→rank0	slip/slip_rate/traction/state/ σ_n (per fault face)	ON (paraview_fault="hdf5")

Key files: - `miniapps/seas/io/paraview_output.hpp` (2394 L) — the `ParaViewOutput<MeshType>` class. - `miniapps/seas/drivers/spatial_dyn_driver.cpp` — construction (L2231-2432), per-step update + save (`paraview_write_lambda`, L2670-2766). - `miniapps/seas/spatial/code/spatial_friction.hpp` `OutputSpec` (L161-189) — config struct. - `miniapps/seas/spatial/code/spatial_friction.cpp` (L1175-1199 output parse; L1490-1492 boundary parse).

1.2 Architecture / data flow

State vector `Q` (`wave_state.hpp`): 9 components, component-major, each block `ndof_total = wave.GetScalarNDof()` long. Indices `SXX=0..SXZ=5`, `VX=6`, `VY=7`, `VZ=8`. So `Q.GetData() + VX*ndof_total` is a contiguous $3 \times \text{ndof_total}$ velocity block.

Per output step (paraview_write(step, time, V_max), L2670): 1. PeekShouldWrite (read-only) decides fault_wants / bulk_wants. 2. If bulk_wants: memcpy 6 stress blocks Q→pv_bulk*_gf; pv_bulk_out→ForceSave. 3. If fault_wants: fill fault projection vectors; memcpy velocity block Q→pv_vel_gf (only when volume_pv_enabled); UpdateFaultFieldsBP5; pv_out→ForceSave (gated on GetVolumeSaveEnabled()); WriteFaultSurfaceVTU. 4. CommitSchedule advances the adaptive schedule state.

Domain fields registered via pv_out→RegisterDomainField(name, gf) → forwards to pv_dc→RegisterField (paraview_output.hpp:591). The data collection (pv_dc_) is a ParaViewDataCollection (VTU/PVD) or ParaViewHDFDataCollection (single .vtkhdf), chosen at construction (L541-555). SetHighOrderOutput(true), SetLevelsOfDetail(order).

Velocity source space (L2310-2316): L2_FECollection(order, dim=3, BasisType::GaussLobatto), ParFiniteElementSpace(..., vdim=3, Ordering::byNODES) → matches the byNODES Q layout. **Stress source space** (L2397-2400): L2_FECollection(order, 3, GaussLobatto), scalar GFs. → Both are **GaussLobatto**, which is the one prerequisite for SubMesh L2 transfer.

The per-step velocity refresh (the snippet the free-surface slice will reuse verbatim, spatial_dyn_driver.cpp:2732-2737) is a single contiguous memcpy of the [VX|VY|VZ] block out of Q — this works precisely because the byNODES, vdim=3 GridFunction data layout equals the component-major Q layout:

```
// byNODES vdim=3 GF data == [all VX dofs][all VY dofs][all VZ dofs]
// == Q's velocity block at offset VX*ndof_total, length 3*ndof_total.
std::memcpy(pv_vel_gf→GetData(),
    Q.GetData() + VX * ndof_total,    // VX = 6
    3 * ndof_total * sizeof(real_t));
```

1.3 Free surface identity

- The free surface is the **zero-traction (natural) BC**, tagged by a mesh boundary attribute: cfg.boundary.natural_attrs (BoundarySpec, spatial_friction.hpp:529-533). TPV meshes: free surface = 1 (natural_attrs = [1]). SAFS .geo: top free = 102.
- Geometry: free surface is at the **top, z = z_max** (driver L1751/L1864 — “free surface at z=z_max”), NOT literally z=0. The slice must select **boundary faces by attribute**, not by a coordinate test. (z=0 in the request = the surface plane; for these meshes that plane is z_max.)

1.4 SubMesh transfer feasibility (verified against MFEM source)

mesh/submesh/ is present **and compiled** (psubmesh.o, ptransfermap.o, ...).

- ParSubMesh::CreateFromBoundary(const ParMesh&, const Array<int>& bdr_attrs) (psubmesh.hpp:83) → codim-1 surface ParMesh; **MPI-collective** (calls ExchangeFaceNbrData + shared-entity discovery). Ranks with no local free-surface faces get an empty local submesh — still must call collectively.
- Field transfer parent↔sub (submesh_utils.cpp:115-165): **L2 (DG) volume → boundary submesh IS supported**, requires parent BasisType::GaussLobatto (ASSERT L117-118 — we satisfy this). Vector vdim=3 byNODES handled (explicit vdim loop L154-159). Value comes from the single adjacent volume element’s trace (GetBdrElementAdjacentElement, L128-131)

— well-defined on a boundary face. H1 also supported.

- ParaViewDataCollection/ParaViewHDFDataCollection accept a ParSubMesh* (SetMesh(Mesh*), datacollection.hpp:310; ParSubMesh inherits ParMesh inherits Mesh). No type guard.
- **No existing CreateFromBoundary use under miniapps/seas/.** The fault writer (fault_vtkhdf_writer.hpp:33-40) deliberately avoided SubMesh — but only because *fault faces are interior* (promoting them to bdr attrs breaks the elasticity attribute contract). The **free surface is already a real boundary attribute**, so that objection does not apply here.

1.5 Restart

The spatial_dyn checkpoint schema does NOT persist ParaView schedule state (driver L38-46: the “6-extra-ParaView-state” form was hypothetical; real schema is smaller). After restart all PV writers start with `last_write_time_ = -1e30` and write on the first post-restart step. The free-surface writer can follow the same fresh-start behaviour; no checkpoint change needed. (Restart must use a distinct `--output-dir`; see memory note.)

1.6 Gotchas

- **Default-ON cost:** CreateFromBoundary is collective and runs once at setup; the transfer map is built once. Per-step cost = one memcpy(s) Q→parent GF + one ParTransferMap apply + a 2D Save (tiny on disk). Memory = the parent source GF(s) — reuse `pv_vel_gf` when volume is enabled, else allocate our own (volume is OFF by default in SAFS, so the slice will usually own its parent velocity GF).
- The parent source GF must live on the **same FE space** the transfer map was built from (L2-GLL, order, vdim=3, byNODES) — mirror the existing `pv_vel_*` setup exactly.
- `mpi_rank` as a slice field is useful for debugging partition seams on the surface.
- Stress at the free surface is ~ 0 for the traction-normal components (zero-traction BC) — velocity is the meaningful ground-motion field; stress is optional/opt-in.

1.7 Open questions (for the plan / user)

1. Which fields on the slice by default — velocity only, or velocity + the 6 stresses?
2. Independent cadence (`paraview_free_surface_dt`) vs. piggyback on the volume/fault schedule?
3. Config key + default value, and whether default-ON should be gated by `paraview_enabled`.